

Колекции от обекти в
полиморфна йерархия. Множествено
наследяване. Диамантен проблем

Семинар 14
Серхан

Хетерогенен контейнер - колекция от обекти от различен тип, но с общ базов клас; реализира се чрез указатели/умни указатели към базовия клас.

clone() - виртуален метод, който всеки наследник имплементира, за да върне точно копие на себе си; позволява копиране без да знае конкретния тип

Множествено наследяване - клас наследява от 2 или повече базови класа
Диамантен проблем - ако два родителски класа имат общ прародител, той се включва 2 пъти в обекта - дублиране на памет

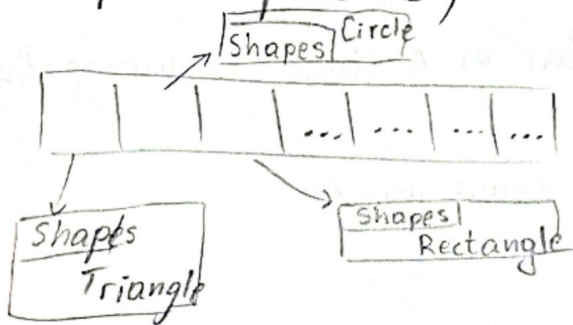
Виртуално наследяване - решението на диамантения проблем

Хетерогенен контейнер

• Искаме колекция, която съдържа различни типове едновременно
Shape shapes[10]; - това е масив от обекти, но всички са Shape, а не Circle/Triangle

Shape* shapes[10];

✓ - масив от указатели към базовия
- всеки сочи към различен наследник



• В базовия клас ще има метод:

```
virtual std::unique_ptr<Shape> clone() const = 0
```

В наследниците се предефинира:

```
std::unique_ptr<Shape> clone() const override
{ return std::make_unique<Circle>(*this); }
```

→ copy констр. на Circle

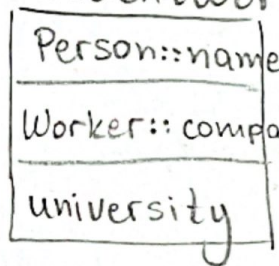
- В класа за колекциите (ShareCollection) пишем copy конструктор и оператор = чрез clone(), а move семантика и деструктор = default.
- Само copy е рѣшен, защото unique_ptr не се копира автоматично и се нундае от clone().

Prototype Design Pattern - нови обекти се създават чрез клониране на вече съществуващ обект(prototype).

Разлика: Factory → създава обект от нулата по зададен тип
 Prototype → създава обект чрез копиране на съществуващ

Множествено наследяване

- обектът съдържа базовите части наредени по рѣра на наследяване

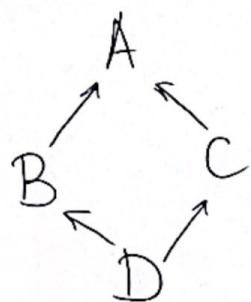


→ ① Наследява Person

→ ② Втория клас, който е базов

→ ③ Своите данни

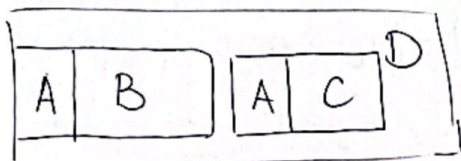
Диамантен проблем



- Класовете B и C наследяват от A. Клас D наследява от B и C.

Резултатът - D съдържа две копия на A.

В паметта:



Виртуално наследяване - гарантира, че общият прародител се включва само веднъж.

- Когато искаме клас да е абстрактен, но всички методи имат имплементация $\Rightarrow$ чисто виртуален деструктор ($=0$)
↓
задължително дефиниция след класа